CIS 4020 · DATA STRUCTURES

My Notes: Red-Black Trees

Insertion and deletion, and how each step matches a 2-3-4 tree

A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are **not** meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim, **tell me**. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker. We are a two-language course, thus I will do my best to show everything in Java **(CIS)** and C/C++ **(CS)**.

1. The rules, and why they work

A red-black tree is an ordinary binary search tree in which every node also stores one extra bit, its color, red or black. The colors must follow these rules:

1. Every node is red or black.
2. The root is black.
3. Every missing child counts as a black leaf. (These are often drawn as small black boxes called NIL leaves. We leave them out of the drawings.)
4. A red node never has a red child. In other words, two reds never appear in a row.
5. Every path from a node down to a missing child passes through the same number of black nodes. That number is the node's **black height**.

Rules 4 and 5 together keep the tree balanced. The shortest possible path from the root is all black, and the longest can at most alternate black and red, so no path is ever more than twice as long as another. That keeps the height at most $2\log_2(n+1)$, so search, insertion, and deletion are all $O(\log n)$.



Figure 1: How colors are drawn in these notes. A black node has a dark fill with white text. A red node has a white fill and a thick dashed outline. A node whose color does not matter for a rule has a light fill and a dotted outline.

A red-black tree is a 2-3-4 tree in disguise

The advanced trees notes showed a red-black tree as a 2-3-4 tree drawn with binary nodes. Read the other way, every black node together with its red children is one 2-3-4 node:

- a black node with no red children is a **2-node**;
- a black node with one red child is a **3-node**;

- a black node with two red children is a **4-node**.

Rule 5 is the 2-3-4 rule that all leaves are on the same level, because only black nodes mark a new 2-3-4 level. Every step of insertion and deletion below is one of the 2-3-4 moves you already know, a split, a borrow, or a merge, carried out with recoloring and rotations.

2. Rotations

A **rotation** changes the shape of a binary search tree without changing its inorder order. It is the only structural tool a red-black tree needs.

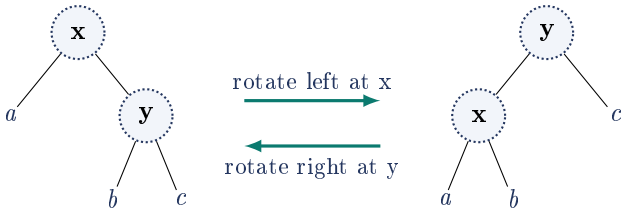


Figure 2: Left and right rotation. On the left, x has subtree a on its left and child y on its right, and y has subtrees b and c. Rotating left at x moves y up into x's place: x becomes y's left child, keeping a, and subtree b moves over to become x's right child. Rotating right at y undoes it. The inorder order, a, x, b, y, c, is the same on both sides.

Only three links change in a rotation, so it takes constant time. The one subtree that changes parents, b, is the one whose values lie between x and y.

3. Insertion

The algorithm

1. Insert the value exactly as in an ordinary binary search tree, as a new leaf.
2. Color the new node **red**. Adding a red node never changes any black height, so rule 5 stays true. The only rule that can break is rule 4: the new node's parent might also be red.
3. If the parent is black, you are done.
4. Otherwise, fix the red-red pair using the cases below, then color the root black.

In the cases, **z** is the red node with the red parent, **p** is its parent, **g** is its grandparent (which must be black, because p is red), and **u** is the **uncle**, g's other child. The uncle's color decides the case. Each case is drawn with p as a left child; when p is a right child, everything is mirrored.

Case 1: the uncle is red, so recolor

Color p and u black and g red. Then g may now have a red parent, so repeat the fix-up with g as the new z. If g is the root, coloring the root black at the end finishes the job.

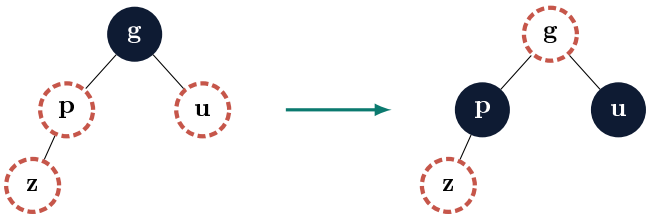


Figure 3: Insertion Case 1. Before: black g with red children p and u, and a new red z under p. After: g is red, p and u are black, and z stays red.

In 2-3-4 terms: black g with two red children is a 4-node, and z is a fourth value trying to join it. Recoloring *splits* the 4-node: p and u become two separate nodes, and g turns red, which means g moves *up* into its parent's 2-3-4 node. That is why the problem can climb, exactly like a split that cascades.

Case 2: the uncle is black and z is an inner child, so rotate at the parent

An *inner* child points toward the middle of the tree: z is a right child while p is a left child, or the mirror image. Rotate at p, toward the outside. Now the old p is the red child of the old z, the shape is a straight line, and Case 3 finishes the job, with the old p playing the role of z.

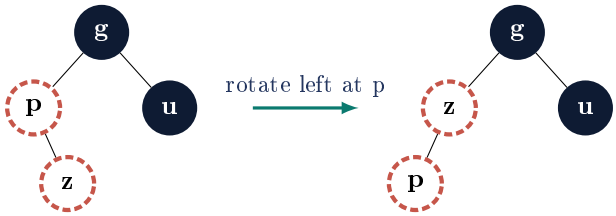


Figure 4: Insertion Case 2. Before: black g with red left child p and black uncle u; z is p's red right child. After rotating left at p: z is g's left child and p is z's left child, both still red, in a straight line.

Case 3: the uncle is black and z is an outer child, so rotate at the grandparent

Color p black and g red, then rotate at g, away from z. Now p is a black node with two red children, z and g. There is no red-red pair left, the black heights are unchanged, and the fix-up is done.

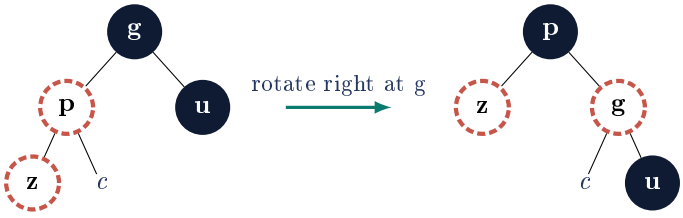


Figure 5: Insertion Case 3. Before: black g with red left child p and black uncle u; z is p's red left child, and c is p's right subtree. After recoloring and rotating right at g: black p with red children z and g, and g has c on its left and u on its right.

In 2-3-4 terms: with a black uncle, black g with one red child p is a 3-node, and z joins it to make a 4-node. Cases 2 and 3 do not split anything. They only redraw the new 4-node the standard way,

with its middle value black on top and the other two values red below it. Nothing moves up, so the fix-up always stops here.

How long can it take? Case 1 can repeat all the way up the tree, but Cases 2 and 3 happen at most once each, at the very end. So insertion does at most two rotations, and $O(\log n)$ recoloring.

Worked example: inserting 60, 55, 85, 25, 45, 15, 20, 30

Insert 60, 55, 85. 60 becomes the root and is colored black. 55 and 85 are red children of a black parent, so nothing needs fixing.

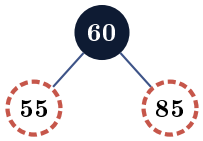


Figure 6: After inserting 60, 55, and 85. Root 60 (black). Level 2: 55 (red), 85 (red).

Insert 25. It goes under 55, and 55 is red. The uncle, 85, is red: **Case 1**. 55 and 85 turn black and 60 turns red. 60 is the root, so it is colored black again.

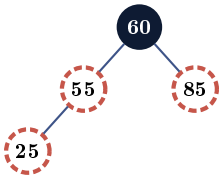


Figure 7: Insert 25, before the fix-up. Root 60 (black). Level 2: 55 (red), 85 (red). Level 3: 25 (red).

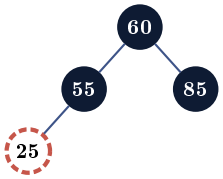


Figure 8: After inserting 25. Root 60 (black). Level 2: 55 (black), 85 (black). Level 3: 25 (red).

Insert 45. It goes to the right of 25, and 25 is red. The uncle is missing, which counts as black, and 45 is an inner child: **Case 2**. Rotate left at 25, which makes 45 the parent of 25. Now the old parent, 25, is an outer child: **Case 3**. Color 45 black and 55 red, then rotate right at 55.

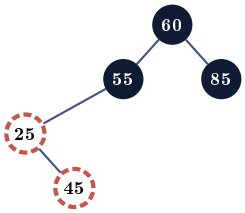


Figure 9: Insert 45, before the fix-up. Root 60 (black). Level 2: 55 (black), 85 (black). Level 3: 25 (red). Level 4: 45 (red).

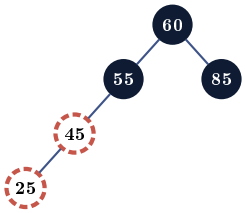


Figure 10: After Case 2, the rotation at 25. Root 60 (black). Level 2: 55 (black), 85 (black). Level 3: 45 (red). Level 4: 25 (red).

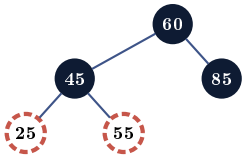


Figure 11: After Case 3, done. Root 60 (black). Level 2: 45 (black), 85 (black). Level 3: 25 (red), 55 (red).

Insert 15. It goes under 25, and 25 is red. The uncle, 55, is red: **Case 1**. 25 and 55 turn black and 45 turns red. 45's parent, 60, is black, so the fix-up stops.

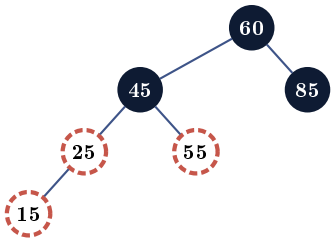


Figure 12: Insert 15, before the fix-up. Root 60 (black). Level 2: 45 (black), 85 (black). Level 3: 25 (red), 55 (red). Level 4: 15 (red).

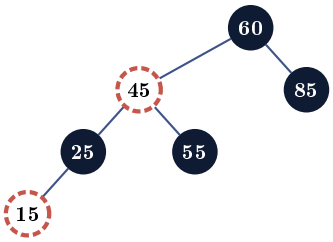


Figure 13: After inserting 15. Root 60 (black). Level 2: 45 (red), 85 (black). Level 3: 25 (black), 55 (black). Level 4: 15 (red).

Insert 20. It goes to the right of 15, and 15 is red. The uncle is missing, so black, and 20 is an inner child: **Case 2**, rotate left at 15. Then **Case 3**: color 20 black and 25 red, and rotate right at 25.

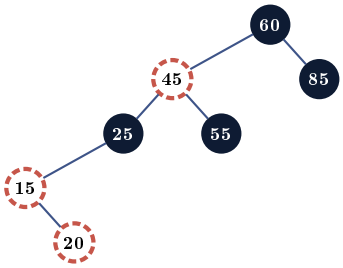


Figure 14: Insert 20, before the fix-up. Root 60 (black). Level 2: 45 (red), 85 (black). Level 3: 25 (black), 55 (black). Level 4: 15 (red). Level 5: 20 (red).

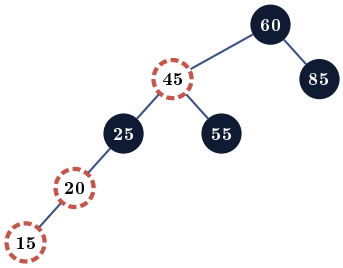


Figure 15: After Case 2, the rotation at 15. Root 60 (black). Level 2: 45 (red), 85 (black). Level 3: 25 (black), 55 (black). Level 4: 20 (red). Level 5: 15 (red).

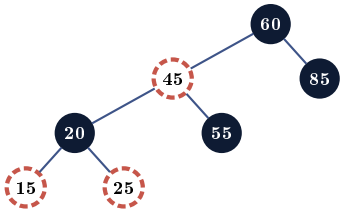


Figure 16: After Case 3, done. Root 60 (black). Level 2: 45 (red), 85 (black). Level 3: 20 (black), 55 (black). Level 4: 15 (red), 25 (red).

Insert 30. It goes to the right of 25, and 25 is red. The uncle, 15, is red: **Case 1**. 15 and 25 turn black and 20 turns red. Now 20 has a red parent, 45, so the fix-up continues with 20 as z. Its uncle, 85, is black, and 20 is an outer child: **Case 3**. Color 45 black and 60 red, and rotate right at 60. 45 becomes the root.

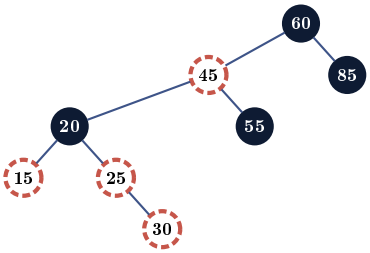


Figure 17: Insert 30, before the fix-up. Root 60 (black). Level 2: 45 (red), 85 (black). Level 3: 20 (black), 55 (black). Level 4: 15 (red), 25 (red). Level 5: 30 (red).

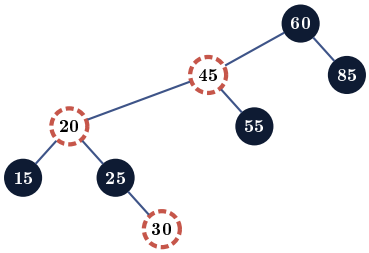


Figure 18: After Case 1: the red pair has moved up to 20 and 45. Root 60 (black). Level 2: 45 (red), 85 (black). Level 3: 20 (red), 55 (black). Level 4: 15 (black), 25 (black). Level 5: 30 (red).

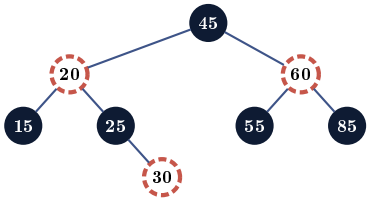


Figure 19: After Case 3, done. Root 45 (black). Level 2: 20 (red), 60 (red). Level 3: 15 (black), 25 (black), 55 (black), 85 (black). Level 4: 30 (red).

In 2-3-4 terms, that last insertion split a 4-node (15, 20, 25) and sent 20 up, where it joined 45 and 60 to make a new 4-node, which Case 3 redrew with 45 in the middle.

4. Deletion

The algorithm

1. Delete the value as in an ordinary binary search tree. If its node has two children, copy the value of its **inorder successor** into it. The node keeps its own color, and the successor's old node is the one that is actually removed. From here on, the color that matters is the color of that removed node.
2. If the node that disappeared was **red**, you are done. No black height changed.
3. If it was **black**, every path through that spot is now one black node short. The node that moved into the spot, call it **x**, is treated as carrying an **extra black**. If x is red, simply color it black and you are done. Otherwise use the cases below to get rid of the extra black.

How much work a deletion takes depends on how many children the node holding the value has.

Table 1: What happens when the node holding the deleted value has no children, one child, or two children.

Children	What happens	Fix-up needed?
None	The node is simply removed.	Only if it was black. Then the empty spot carries the extra black, and Cases 1 to 4 apply.
One	The red child moves up into the node's place and turns black.	Never.
Two	The successor's value is copied in, and the successor's node is removed instead.	Depends on the successor. The successor never has a left child, so removing it is the "None" or "One" row.

A node with exactly one child is always black, and its child is always red with no children of its own. The missing side has no black nodes below it, so by rule 5 the other side cannot have any either, which makes the child red, and a red child cannot have a red parent. In the starting tree of the deletion example below, 25 is black with the single red child 30; deleting 25 would just move 30 up and color it black.

Think of deletion in two stages. In the **removal** stage, a value may be copied up and a node is removed lower down, but no interior node changes color. In the **fix-up** stage, which happens only if the removed node was black, the recolors and rotations of Cases 1 to 4 start at the removed spot and can change the colors of interior nodes on the way up. Insertion works the same way: placing the new red node changes nothing above it, and the fix-up's recolors and rotations can change interior colors all the way to the root.

x may be a missing child, which counts as black. In the cases, **w** is x's **sibling**. The sibling and its children decide the case. Each case is drawn with x as a left child; when x is a right child, everything is mirrored. "Near" means the sibling's child closer to x, and "far" means the other one.

Case 1: the sibling is red

Color w black and the parent red, then rotate at the parent toward x. x now has a new sibling, which is black, so go on to Case 2, 3, or 4.

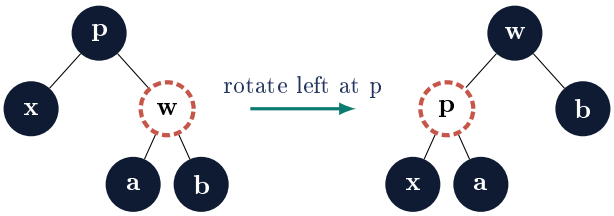


Figure 20: Deletion Case 1. Before: black parent p with black x on the left and red sibling w on the right; w has black children a and b. After: black w on top, its left child p is red with children x and a, and b stays on the right. x's new sibling is a, which is black.

In 2-3-4 terms: nothing happens. p and red w are one 3-node drawn with its red value on the far side. The rotation redraws the same 3-node the other way, so that x's real 2-3-4 sibling ends up next to x.

Case 2: the sibling is black, and both of its children are black

Color w red, and move the extra black up to the parent. If the parent is red, color it black and you are done. If the parent is black, the parent becomes the new x and the fix-up repeats one level higher.

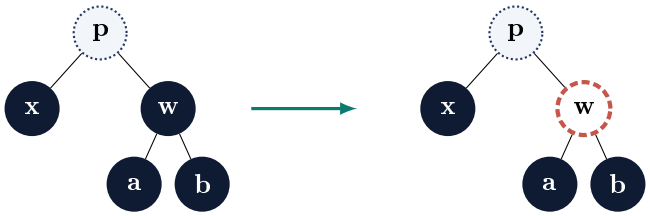


Figure 21: Deletion Case 2. Before: parent p, of either color, with black x and black sibling w; w's children a and b are both black. After: w is red, and the extra black has moved up to p.

In 2-3-4 terms: this is a **merge**. x's 2-3-4 node is empty and its sibling is a 2-node with nothing to lend, so the sibling and a value from the parent combine. Turning w red joins it to the parent's node. If the parent was a 2-node, the parent is now empty, which is why the problem can climb, exactly like a merge that cascades in a 2-3 tree.

Case 3: the sibling is black, its near child is red, and its far child is black

Color the near child black and w red, then rotate at w, away from x. Now x's sibling is the old near child, and its far child is red, so go on to Case 4.

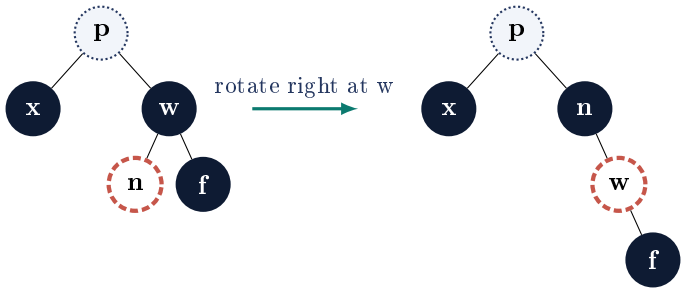


Figure 22: Deletion Case 3. Before: parent p, of either color, with black x and black sibling w; w's near child n is red and its far child f is black. After: n is black and is x's new sibling, and w is n's red right child, with f below it. The far child of x's new sibling, w, is now red.

Case 4: the sibling is black and its far child is red

Give w the parent's color, color the parent and the far child black, and rotate at the parent toward x. The extra black is gone, and the fix-up is done.

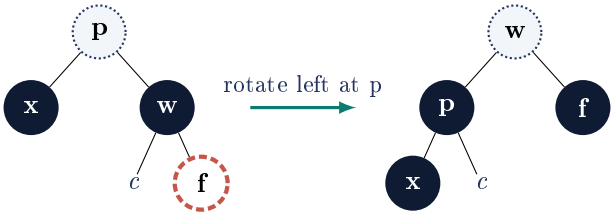


Figure 23: Deletion Case 4. Before: parent p, of either color, with black x and black sibling w; w's far child f is red, and c is w's near subtree. After: w has taken p's place and p's old color; p is black with x and c below it; f is black.

In 2-3-4 terms: Cases 3 and 4 are a **borrow**. The sibling is a 3-node or a 4-node, so it has a value to spare. A value from the parent moves down to x's side, and a value from the sibling moves up to replace it. Case 3 only arranges the sibling so that the spare value is on the far side, where Case 4's rotation can use it. Because nothing is merged, the fix-up always stops after a borrow.

How long can it take? Case 2 can climb all the way to the root, but Cases 1, 3, and 4 each happen at most once in a row before the fix-up ends. So deletion does at most three rotations, and $O(\log n)$ recoloring.

Worked example: deleting 85, 45, 15, 60, 20

We start from the tree the insertion example built.

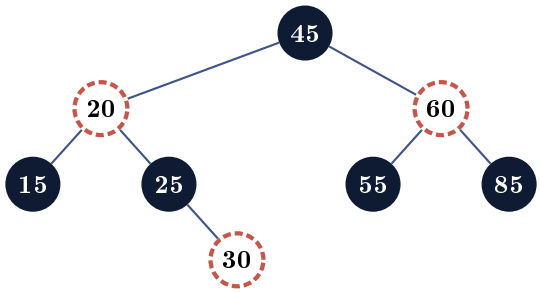


Figure 24: The starting tree for the deletion example. Root 45 (black). Level 2: 20 (red), 60 (red). Level 3: 15 (black), 25 (black), 55 (black), 85 (black). Level 4: 30 (red).

Delete 85: Case 2. 85 is black and has no children, so a missing, black child takes its place and carries the extra black. Its sibling, 55, is black, and both of 55's children are missing, so black: Case 2. 55 turns red, and the extra black moves up to the parent, 60. 60 is red, so it is colored black and the fix-up ends.

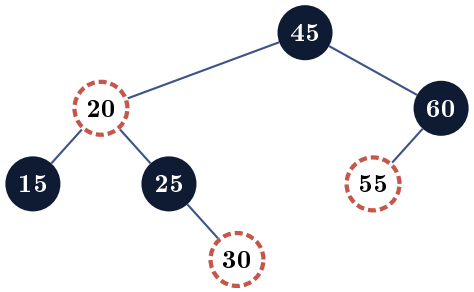


Figure 25: After deleting 85. Root 45 (black). Level 2: 20 (red), 60 (black). Level 3: 15 (black), 25 (black), 55 (red). Level 4: 30 (red).

Delete 45: a successor value that removes a red node. 45 has two children, so the value of its inorder successor, 55, is copied into 45's node, and that node keeps its color, black. 55's old node is the one actually removed, and it was red, so no black count changed and there is nothing to fix.

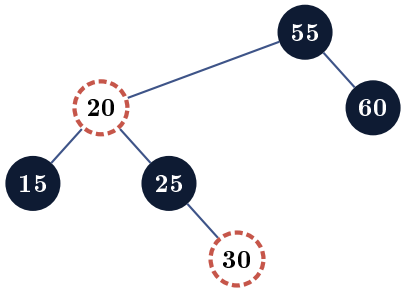


Figure 26: After deleting 45. Root 55 (black). Level 2: 20 (red), 60 (black). Level 3: 15 (black), 25 (black). Level 4: 30 (red).

Delete 15: Case 4. 15 is black and has no children, so a missing child takes its place with an extra black. Its sibling, 25, is black, and 25's far child, 30, is red: Case 4. 25 takes its parent's color, red; the parent 20 and the far child 30 turn black; and the tree rotates left at 20.

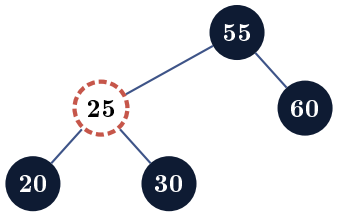


Figure 27: After deleting 15. Root 55 (black). Level 2: 25 (red), 60 (black). Level 3: 20 (black), 30 (black).

Delete 60: Case 1, then Case 2. 60 is black and has no children, so a missing child takes its place with an extra black. Its sibling, 25, is red: Case 1. 25 turns black, the parent 55 turns red, and the tree rotates right at 55. Now x's sibling is 30, which is black with no children: Case 2. 30 turns red, and the extra black moves up to 55. 55 is red, so it is colored black and the fix-up ends.

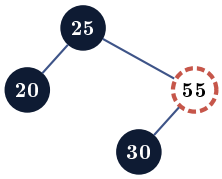


Figure 28: After Case 1, the rotation at 55. Root 25 (black). Level 2: 20 (black), 55 (red). Level 3: 30 (black).

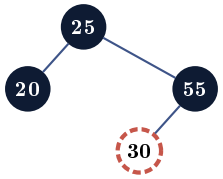


Figure 29: After Case 2, done. Root 25 (black). Level 2: 20 (black), 55 (black). Level 3: 30 (red).

Delete 20: Case 3, then Case 4. 20 is black and has no children, so a missing child takes its place with an extra black. Its sibling, 55, is black; its near child, 30, is red, and its far child is missing, so black: Case 3. 30 turns black and 55 red, and the tree rotates right at 55. Now x's sibling is 30, whose far child, 55, is red: Case 4. 30 takes the parent's color, black; the parent 25 and the far child 55 turn black; and the tree rotates left at 25.

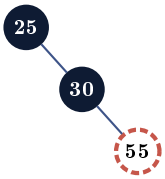


Figure 30: After Case 3, the rotation at 55. Root 25 (black). Level 2: 30 (black). Level 3: 55 (red).

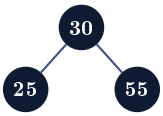


Figure 31: After Case 4, done. Root 30 (black). Level 2: 25 (black), 55 (black).

5. The book's view: top-down

The textbook explains red-black trees through the 2-3-4 algorithms you already know, so it does its work on the way *down*, the same way preemptive splitting does. The bottom-up method above is the one you will find in most code and most other sources. Both keep the same rules and the same $O(\log n)$ guarantees, but they can build different trees from the same values.

- **Top-down insertion.** On the way down, whenever you meet a black node with two red children, which is a 4-node, do a **color flip**: the node turns red and its two children turn black. That is a preemptive split, with the middle value moving up. If the flip leaves two reds in a row, one or two rotations fix it, the same rotations as Cases 2 and 3. Then insert the new red leaf. If its parent is red, one more rotation finishes, and the fix-up never has to climb back up.
- **Top-down deletion.** This follows the 2-3-4 deletion from the advanced trees notes: on the way down, every 2-node is turned into a 3-node or 4-node before the search enters it. In red-black terms, a color flip in reverse merges, and a rotation borrows. The book describes this at the

same general level and leaves the details as an exercise.

6. The code

The code for these notes is in separate files: `RedBlackTree.java` and `RedBlackDriver.java` for Java **(CIS)**, and `RedBlackTree.h`, `RedBlackTree.cpp`, and `main.cpp` with a `Makefile` for C/C++ **(CS)**. In C++, `delete` is a keyword, so the method is named `remove`. The code is the bottom-up algorithm from these notes. Missing children point to one shared black **sentinel** node, NIL, instead of `null` or `nullptr`, so the fix-ups can ask for the color of a missing child without a special case every time. When a node has two children, the code moves the successor's node into the deleted node's position and gives it that node's color, instead of copying the value; the resulting tree is exactly the same.

The driver runs both worked examples and prints the tree after every operation, so you can compare each step with the figures. Both versions were run on the two worked examples, with every tree matching the figures, and on 3,000 random mixes of insertions and deletions, checking every red-black rule after every operation.

7. Common mistakes

- **Inserting a black node.** New nodes are always red. A black node would break rule 5 everywhere along its path, which is much harder to fix than a red-red pair.
- **Looking at the sibling instead of the uncle during insertion.** The uncle, the parent's sibling, decides the insertion case. The sibling decides the deletion case.
- **Forgetting the mirror image.** Every case has a left and a right version. Swap every left for right, and every rotate left for rotate right.
- **Forgetting that missing children are black.** A missing uncle or a missing child of the sibling counts as black.
- **Using the wrong color to decide whether to fix up.** When a successor's value is copied up, the color that matters is the color of the successor's node, the one actually removed, not the color of the node whose value was deleted.
- **Stopping after Case 1 or Case 3 of deletion.** Neither one finishes the fix-up. Case 1 always leads to Case 2, 3, or 4, and Case 3 always leads to Case 4.
- **Forgetting to color the root black** at the end of insertion.

8. Where this lives in the textbooks

- **Java, 3rd edition (Prichard and Carrano).** Chapter 13, Advanced Implementations of Tables, Section 13.1, under Red-Black Trees.
- **C/C++, 8th edition (Carrano and Henry).** Chapter 26, Advanced Search Trees, Section 26.2, Red-Black Trees.
- The bottom-up cases and the sentinel follow the classic presentation in Cormen, Leiserson, Rivest, and Stein, *Introduction to Algorithms*, the chapter on red-black trees, which is where most online explanations come from.